4. Interfaces graphiques



4. Interfaces graphiques : les Styles

- **Un style est une collection de propriétés qui spécifie le design d'une vue ou d'un élément, il peut spécifier différentes propriétés :**
  - Hauteur
  - Largeur
  - Couleur du text
  - Taille du texte
  - Etc
- **Un style est défini dans un fichier ressource XML enregistré sous res/values/ avec le nom : style.xml**

4. Interfaces graphiques : les Styles

- **Sous le fichier de ressource XML res/values/style.xml on définit les deux styles suivants:**

```
<style name="monstyle1" >
    <item name="android:layout_width">match_parent</item>
    <item name="android:layout_height">wrap_content</item>
    <item name="android:textcolor">@color/red</item>
    <item name="android:textSize">40sp</item>
</style>
```

```
<style name="monstyle2" parent="Theme.AppCompat.Light">
    <item name="android:layout_width">match_parent</item>
    <item name="android:layout_height">wrap_content</item>
    <item name="android:color">@color/bleu</item>
    <item name="android:textSize">20sp</item>
</style>
```


4. Interfaces graphiques : les Styles

- **On applique les deux styles sur les widgets:**

```
<TextView
    style="@style/monstyle1"
    android:text="Text1"/>


<TextView
    style="@style/monstyle2"
    android:text="Text2"/>
```

4. Interfaces graphiques



4. Interfaces graphiques : les Thèmes

- Les thèmes Android sont appliqués sur les Layouts. Autrement dit, c'est un style pour les Layouts.
- Exemple :
  - Dans un Layout android:layout_[width|height] sont obligatoires, on peut, par exemple, prévoir un thème initialisant l'ensemble de ces valeurs à mach_parent.
  - En ajoutant une couleur claire en arrière-plan (background) des Layouts
  - On commence par créer un style nommé Theme.Custom héritant du thème Android AppTheme.

4. Interfaces graphiques : les Thèmes

```
<?xml version="1.0" encoding="utf-8"?>
<resources>

        <style name="Theme.Custom" parent= "@android:style/AppTheme">
            <item name="android:layout_height">match_parent</item>
            <item name="android:layout_width">match_parent</item>
            <item name="android:background">@color/claire</item>
        </style>

</resources>
```


4. Interfaces graphiques : les Thèmes

- Pour appliquer le thème crée, on ajoute dans le fichier AndroidManifest.xml la ligne suivante :

```
<application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.Custom"
        ..................>
.....................
</application>
```

- A ce niveau tous les éléments de l'application vont utiliser le même thème.
- Il est possible de personnalisé quelque élément du thème par des attribut de style

4. Interfaces graphiques



4. Interfaces graphiques : Les Menus

- Trois types de menus
  - Option menu : un ensemble d'options principales pour une application, les utilisateurs peuvent sélectionner l'une des options pour effectuer une action.
  - Context Menu : est un menu flottant qui s'affiche pour répondre à un appui long, la valeur par défaut est de 500 millisecondes. Context Menu peut contenir de nombreux éléments de Menu Item et Sub
  - Popup menu : est un menu flottant, vous pouvez le créer et l'ancrer dans n'importe quelle View. En termes d'interface et d'utilisation, il n'est pas différent d'un Context Menu.
- Les menus sont placés différemment selon la version d'Android :
  - En bas : jusqu'à Android 3.2.2 (API <= 13)
  - En haut à droite à partir d'Android 4.0.1 (API >= 14)

4. Interfaces graphiques : Les Menus : Code XML

- Il faut créer un fichier : res/menu/menu.xml

```
<menu xmlns:android="http://schemas.android.ccm/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">
    <item android:id="@+id/m1"
        android:title="File">
        <menu>
            <item android:id="@+id/m11"   android:title="new"/>
            <item android:id="@+id/m12"   android:title="open"/>
            <item android:id="@+id/m13"   android:title="save"/>
        </menu>
    </item>
    <item android:id="@+id/m2"
        android:title="Edit">
        <menu>
            <item android:id="@+id/m21"   android:title="copier"/>
            <item android:id="@+id/m22"   android:title="couper"/>
            <item android:id="@+id/m23"   android:title="coller"/>
        </menu>
    </item>
    <item android:id="@+id/m3"            android:title="Affiche"/>
</menu>
```


4. Interfaces graphiques : Les Menus : Design





4. Interfaces graphiques : Les Menus : Code Java

- Les menus peuvent être différents selon les activités :
- On utilise :
  - La méthode onCreateOptionMenu
  - L'objet : MenuInflater
    - Les inflaters sont des objets qui créent des vues à partir de fichiers XML

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.menu,menu);
    return true;
}
```


4. Interfaces graphiques : Les Menus : Code Java

```
//Méthode qui se déclenchera au clic sur un item
public boolean onOptionsItemSelected(MenuItem item) {
      switch (item.getItemId()) {
      case R.id.option: .....; return true;
      case R.id.favoris: .....; return true;
      case R.id.stats: .....; return true;
      case R.id.quitter: finish(); return true;
}
return false;}
```


4. Interfaces graphiques : Les Menus

- Android Menu



Context Menu

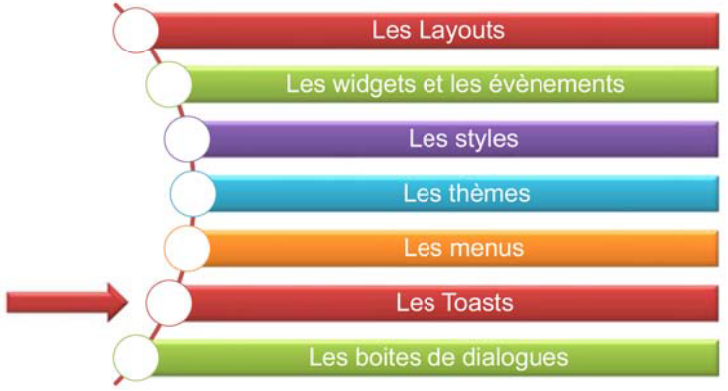
https://devstory.net/12615/android-contextmenu



Popup Menu

https://devstory.net/12631/android-popupmenu

4. Interfaces graphiques



4. Interfaces graphiques : Les Toasts

- Android Toast est une petite notification que l'application envoie aux utilisateurs, elle apparaît près du bas de l'écran (par défaut) et disparaît automatiquement lorsque le temps est écoulé.
- Il est possible de personnaliser un Toast avec la bibliothèque externe Toasty

```
Toast.makeText(this, "Bonjour mes Amis", Toast.LENGTH_SHORT).show();
```

Avec :

- This est le context de l'activity en cours : getapplicationcontext().
- Le message à afficher.
- La durée d'affichage : avec une durée :
  - Toast.LENGTH_SHORT : 2 secondes.
  - Toast.LENGTH_LONG : 5 secondes

4. Interfaces graphiques : Les Toasts

- Toast Position : Par défaut, Toast s'affiche près du bas de l'écran, centré horizontalement. En utilisant la méthode setGravity (), vous pouvez modifier sa position d'affichage.

```
Toast toast = Toast.makeText(MainActivity.this, "This is a message!", Toast.LENGTH_SHORT);
toast.setGravity(Gravity.LEFT | Gravity.TOP, 20, 30);
toast.show();
```

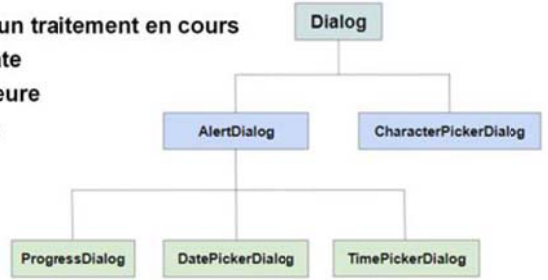
```
Toast toast = Toast.makeText(MainActivity.this, "This is a message!", Toast.LENGTH_SHORT);
toast.setGravity(Gravity.Right, 20, 30);
toast.show();
```


4. Interfaces graphiques



4. Interfaces graphiques : Les boites de dialogues

- Les boites de dialogues avec Android permettent d'afficher des informations en mode modales avec un ensemble de bouton configurable pour interagir avec l'utilisateur
- Les boites de dialogues héritent de la classe android.app.Dialog
- Android fournit des boites de dialogues prédéfinies :
  - AlerteDialog : alerte
  - ProgressDialog : information sur un traitement en cours
  - DatePickerDialog : choix d'une date
  - TimePickerDialog : choix d'une heure
  - ... extends Dialog : personnalisée



4. Interfaces graphiques : Les boites de dialogues

- **AlertDialog est une sous-classe directe de Dialog, elle a une zone d'en-tête, une zone de contenu et à 1, 2 ou 3 boutons.**



4. Interfaces graphiques : Les boites de dialogues

- **Les étapes d'utilisation d'un objet de type AlertDialog :**
  - **Instanciation d'un objet de type AlertDialog**
    - **AlertDialog.Builder builder = new AlertDialog.Builder(MainActivity.this);**
  - **Configuration de l'objet et le personnaliser :**
    - Spécification de l'icône : **.setIcon(R.mipmap.*ic_launcher*)**
    - Spécification du Titre : **.setTitle(R.string.*Dialogue_titre_erreur*)**
    - Spécification message : **.setMessage(R.string.*Dialogue_msg_erreur*)**
  - **Spécification des boutons :**

```
.setPositiveButton(R.string.Bt_dialog_ok, new DialogInterface.OnClickListener() {
public void onClick(DialogInterface dialog, int id) {
//Traitement à faire
        }
})
```


4. Interfaces graphiques : Les boites de dialogues

```
.setNegativeButton(R.string.Bt_dialog_ok, new DialogInterface.OnClickListener() {
public void onClick(DialogInterface dialog, int id) {
//Traitement à faire
}
})
.setNeutralButton(R.string.Bt_dialog_ok, new DialogInterface.OnClickListener() {
public void onClick(DialogInterface dialog, int id) {
//Traitement à faire
}
})
```


4. Interfaces graphiques : Les boites de dialogues

- **Création de l'objet AlertDialog**

```
builder.create()
```

- **affichage**

```
builder.show()
```

- **Méthode utiles :**
  - **dialog.cancel();** permet de fermer la boite de dialog affichée
  - **nom_activity.this.finish();** Quitter l'application

4. Interfaces graphiques : Les boites de dialogues

- Android TimePickerDialog



Spinner Mode



Clock Mode

https://devstory.net/12741/android-timepickerdialog

4. Interfaces graphiques : Les boites de dialogues

- Android DatePickerDialog



Spinner Mode



Calender Mode

https://devstory.net/12739/android-datepickerdialog

4. Interfaces graphiques : Les boites de dialogues

- Android ProgressDialog

